Шинжлэх Ухаан Технологийн Их Сургууль
Мэдээлэл, Холбооны Технологийн Сургууль



F.CSM302 Үйлдлийн систем

UNIX Shell

Бие даалт №1

Судалгаа, хэрэгжилт, туршилтын тайлан

Шалгасан багш:	Г.Ганбат
Хуваарилалтын код:	IV-2-1
Гүйцэтгэсэн оюутнууд:	Б.Билгүүн /B242270035/ Г.Анужин /B241940083/ Э.Чагдарсүрэн /B160910044/ Ц.Энх-Од /B231890017/ Г.Маралгуа /B210930821/ Т.Саруулзаяа /B232230057/

Улаанбаатар хот
2026 он

Гарчиг

Гарчиг **2**

Хураангуй **4**

1. Удиртгал **5**

1.1 Зорилго 5

1.2 Ач холбогдол 5

1.3 Хамрах хүрээ 5

1.4 Ажлын хуваарилалт 5

2. Сурах бичгийн шаардлага **5**

3. Онолын үндэс **6**

3.1 Shell-ийн read–eval–print давталт 6

3.2 Процесс үүсгэх системийн дуудлагууд 6

3.3 Файл тодорхойлогч ба дахин чиглүүлэлт 7

3.4 Pipe ба олон шатлалтай холбоо 7

3.5 Дотоод (built-in) командууд 8

3.6 Zombie процесс 8

4. Системийн зохиомж **8**

4.1 Бүрэлдэхүүн хэсгүүд 8

4.2 Өгөгдлийн урсгал 9

4.3 Үйл явдлын бүртгэл 9

4.4 Процессын төлвийн загвар 9

5. Хэрэгжилт **10**

5.1 Хэл, сан, орчин 10

5.2 Хязгаарууд 10

5.3 Parser 11

5.4 Командын гүйцэтгэл 11

5.5 Дотоод командууд, history, background цуглуулалт 11

5.6 Сервер ба session 11

5.7 UI-ийн хэрэгжилт 12

6. Туршилт ба UI дүрслэл **12**

6.1 Юуг өөрчилж турших вэ 12

6.2 Юуг ажиглах вэ 12

6.3 UI-ийн бодит дэлгэцийн зураг 12

7. Туршилтын арга **16**

7.1 Орчин 16

7.2 Автомат тестийн арга 16

7.3 Гар туршилтын арга 16
8. Үр дүн 16
8.1 Автомат тестийн үр дүн 16
8.2 Гар туршилтын үр дүн 18
8.3 Нэмэлт туршилт (захын тохиолдол) 19
9. Хэлэлцүүлэг 20
9.1 Үр дүнгийн тайлбар 20
9.2 Хязгаарлалт 21
10. Дүгнэлт 21
Ашигласан материал 21
Хавсралт А. Ажлын төлөвлөгөө 23

Хураангуй

Энэ ажлын хүрээнд Operating System Concepts (10th edition) сурах бичгийн 3-р бүлгийн Programming Project 1 буюу UNIX Shell төслийг C хэл дээр хэрэгжүүллээ. Хэрэгжүүлсэн `osh` shell нь хэрэглэгчийн оруулсан командыг `fork()` системийн дуудлагаар үүсгэсэн child процесст `execvp()`-ээр ажиллуулж, `waitpid()`-ээр хүлээдэг. Командын төгсгөлд `&` тэмдэгт байвал хүлээлгүйгээр background-д ажиллуулж, `!!` командаар хамгийн сүүлд ажилласан командыг давтаж, `history` командаар сүүлийн 10 командыг дугаартай нь харуулна. Оролт, гаралтыг `<` ба `>` тэмдэгтээр файл руу чиглүүлж (`open()` ба `dup2()`), командуудыг `|` тэмдэгтээр pipe-аар холбоно (`pipe()` ба `dup2()`). Хэрэглэгч хэдэн ч шатлалтай pipe (дээд тал нь 16) ашиглаж болно.

Төслийн UI дүрслэл болох Process tree-г Python локал сервер болон HTML хуудсаар хэрэгжүүлсэн. Сервер нэг `osh` процессыг тасралтгүй барьж, хэрэглэгчийн команд бүрийг түүний stdin-д дамжуулна. Shell нь процесс бүр үүсэх, дуусах үед `OSH_LOG` файлд JSON мөр бичдэг бөгөөд UI энэ бүртгэлээс командын дугаар, PID, төлөв бүхий мод байгуулна. Background процессыг дууссан боловч shell хараахан `wait` хийгээгүй үед `zombie` төлөвтэй харуулдаг нь онолын ойлголтыг шууд ажиглах боломж олгодог.

Туршилтыг Xubuntu 25.10 (VirtualBox), gcc 15.2.0 орчинд хийж, `make test` командаар ажиллах 22 автомат тест бүгд тэнцсэн (22/22). Хөрвүүлэлтэд анхааруулга гараагүй. UI-г хөтөч дээр гараар туршиж, pipe, redirection, background, history, `cd`, алдааны тохиолдлуудыг screenshot-оор баримтжуулсан. Хашилт, wildcard, орчны хувьсагчийн тэлэлт, `>>`, `2>`, бүрэн job control хэрэгжээгүй. UI болон сервер автомат тестээр хамрагдаагүй гэдгийг хязгаарлалтад тусгасан.

1. Удиртгал

1.1 Зорилго

Shell бол хэрэглэгч болон үйлдлийн системийн цөмийн хоорондох гол интерфейс юм. Хэрэглэгч терминалд нэг мөр команд бичихэд цаана процесс үүсгэх, шинэ программ ачаалах, процессыг хүлээх, файл тодорхойлогчийг дахин чиглүүлэх, процессууд хоорондын өгөгдлийн урсгалыг pipe-аар холбох зэрэг хэд хэдэн системийн дуудлага дараалан ажилладаг. Энэ ажлын зорилго нь эдгээр дуудлагыг өөрсдөө хэрэгжүүлж, процессын удирдлагын үндсэн ойлголтыг практикт ойлгох, мөн үүссэн процессуудыг бодит PID-тай нь харуулах UI-тай холбож, онолын ойлголтыг ажиглагдахуйц болгох явдал юм.

1.2 Ач холбогдол

Fork, exec, wait гэсэн гурван дуудлага нь UNIX төст үйлдлийн системийн процессын загварын үндэс. Эдгээрийг ойлгосноор процесс эцэг эхийн холбоо, zombie ба orphan процесс, exit status, stdin/stdout-ийн дахин чиглүүлэлт зэрэг ойлголтууд хоорондоо хэрхэн холбогдохыг харж болно. Мөн `ls | wc -l` гэх мэт өдөр тутмын командууд цаанаа хэд хэдэн процесс, pipe-аар хэрэгждэгийг Process tree-ээр нүдээр харж баталгаажуулна.

1.3 Хамрах хүрээ

Хамрах хүрээнд сурах бичгийн үндсэн шаардлага (child процесс, `&`, `!!`, `< >`, `|`), хэдэн ч шатлалтай pipe, `cd` ба `exit` дотоод командууд, нэг тасралтгүй shell session барьдаг локал сервер, Process tree UI, автомат тест, Report болон Manual орно. Хамрахгүй зүйлд хашилт, wildcard (`*`), орчны хувьсагчийн тэлэлт (`$HOME`), `>>` ба `2>` чиглүүлэлт, бүрэн job control (`fg`, `bg`, `Ctrl+Z`), UI болон серверийн автомат тест, Windows дээрх native ажиллагаа багтана.

1.4 Ажлын хуваарилалт

Ажлыг зургаан гишүүнтэй баг хамтран гүйцэтгэсэн. Гишүүд эх код, тест, UI, тайлан, manual-ыг хамтран бэлтгэж, GitHub repository-д commit хийсэн бөгөөд гишүүн бүрийн оруулсан өөрчлөлт repository-ийн commit түүхэнд харагдана. Эх код, тест, баримт бичиг бүгд https://github.com/StrAVengeR/IV-2-1-UNIX-Shell repository-д байрлана.

2. Сурах бичгийн шаардлага

Сурах бичгийн 3-р бүлгийн Programming Project 1 нь командын интерпретатор (shell) бичихийг шаарддаг. Shell нь `osh>` prompt-ыг хэвлэж, хэрэглэгчийн командыг уншиж, child процесс үүсгэн ажиллуулна. Үндсэн шаардлагуудыг доорх хүснэгтэд нэгтгэж, тус бүрийг хэрхэн хангасныг харууллаа.

Хүснэгт 2.1. Шаардлага ба хэрэгжилт

№	Сурах бичгийн шаардлага	Манай хэрэгжилт	Хаана
1	Командыг child процесст ажиллуулах: `fork()`, `execvp()`, parent нь `wait()`	Шатлал бүрд `fork()`, child дотор `execvp()`. Foreground үед parent `waitpid()`-ээр хүлээнэ.	`run()`
2	Командын төгсгөлд `&` байвал parent child-ийг хүлээхгүй	Background бол `waitpid()` дуудахгүй, `[bg] pid N` хэвлээд prompt руу буцна. Дууссаныг дараагийн prompt-ийн өмнө `waitpid(-1, WNOHANG)`-аар цуглуулна.	`run()`, `reap_background()`
3	History: `!!` нь хамгийн сүүлийн командыг давтана. Түүх хоосон бол алдаа мэдээлнэ.	`hist[]` массивт командыг хадгална. Хоосон үед `No commands in history.` хэвлэнэ. `history` нь сүүлийн 10 командыг дугаартай нь харуулна.	`main()`
4	Оролт гаралтыг чиглүүлэх: `>` ба `<`	Child дотор `open()` хийж `dup2()`-оор fd 0 эсвэл 1-ийг сольно.	`run()`
5	Pipe: хоёр командын хооронд `pipe()` ба `dup2()`	Шатлал бүрийн хооронд `pipe()` үүсгэнэ. Хэдэн ч шатлалыг дэмжинэ (16 хүртэл).	`run()`
6	Буруу болон захын оролтыг ойлгомжтой мэдээлэх (Бие даалт 1-ийн нийтлэг шаардлага)	Синтакс алдаа, байхгүй команд, байхгүй файл, хэт урт мөр, хэт олон шатлалд тус бүрд тодорхой мессеж гаргана.	`parse()`, `run()`
7	Өөр оролт, параметрээр дахин турших боломж	UI-д дурын команд бичиж ажиллуулна, Reset хийж шинэ session эхлүүлнэ.	`ui/`
8	UI дүрслэл: Process tree	Командын дугаар, PID, төлөв бүхий мод. Төлвийг OS-оос шалгана.	`ui/server.py`, `ui/index.html`

3. Онолын үндэс

3.1 Shell-ийн read–eval–print давталт

Shell нь үндсэн нь нэг л давталт юм. Эхлээд prompt хэвлэж, хэрэглэгчээс нэг мөр уншина. Дараа нь тэр мөрийг token болгон задалж (parsing), ямар команд, ямар аргумент, ямар чиглүүлэлт, pipe байгааг тодорхойлно. Тэгээд процесс үүсгэж командыг ажиллуулна. Төгсгөлд үр дүнг (гаралт, exit status) хэвлээд дахин prompt руу буцна. Манай `main()` функц яг ийм бүтэцтэй бөгөөд `exit` бичих эсвэл оролт дуусахад (EOF) давталтаас гарна.

3.2 Процесс үүсгэх системийн дуудлагууд

UNIX-д шинэ программыг ажиллуулахдаа процессыг хоёр алхмаар үүсгэдэг. Эхлээд `fork()` дуудлага нь дуудсан процессын (parent) бараг яг хуулбар болох шинэ процессыг (child) үүсгэнэ. `fork()` нь parent-д child-ийн PID-г, child-д 0-г буцаадаг тул хоёр процесс өөрсдийн хэн болохыг энэ утгаар ялгана. Дараа нь child нь `execvp()` дуудаж өөрийн санах ойн агуулгыг шинэ программаар бүрэн сольдог. Командын нэрийг PATH хувьсагчаас хайдаг тул `ls` гэж бичихэд `/usr/bin/ls` олдоно. `execvp()` амжилттай бол хэзээ ч буцаж ирдэггүй. Буцаж ирсэн нь алдаа гэсэн үг (жишээ нь команд олдоогүй).

Parent нь `waitpid()` дуудаж child дуустал хүлээж, child-ийн exit status-ыг авна. Манай shell командыг амжилтгүй олдохгүй бол child `127` кодоор гарна. Хэрэв child дохиогоор (signal) зогссон бол exit код нь 128 дээр дохионы дугаарыг нэмсэн утга болдог. Эдгээр нь UNIX-ийн уламжлалт тогтоолцоо бөгөөд UI-д улаан `exited 127` гэж харагдана.

Хүснэгт 3.1. Ашигласан системийн дуудлагууд

Дуудлага	Үүрэг	Манай кодод
`fork()`	Parent-ийн хуулбар child процесс үүсгэнэ. Parent-д child PID, child-д 0 буцна.	Шатлал бүрд нэг удаа
`execvp(file, argv)`	Child-ийн программыг шинэ программаар солино. PATH-аас хайна.	Child дотор, чиглүүлэлтийн дараа
`waitpid(pid, &st, opt)`	Тодорхой child дуустал хүлээж, status авна. `WNOHANG` нь хүлээхгүйгээр шалгана.	Foreground, background цуглуулалт
`pipe(fd)`	Нэг чиглэлтэй байт урсгал. `fd[0]` унших, `fd[1]` бичих төгсгөл.	Шатлал хоорондын холбоо
`dup2(a, b)`	Fd `b`-г fd `a`-тай ижил файлыг заадаг болгоно.	stdin/stdout дахин чиглүүлэх
`open(path, flags)`	Файл нээж fd буцаана.	`<` ба `>` чиглүүлэлт
`chdir(path)`	Дуудсан процессын ажлын хавтсыг солино.	`cd` дотоод команд

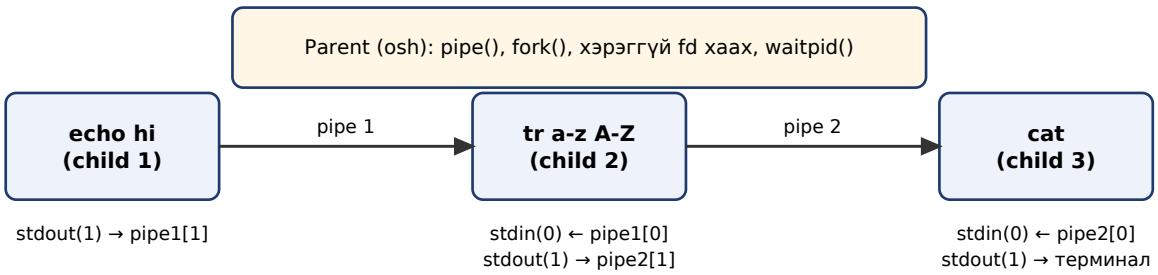
3.3 Файл тодорхойлогч ба дахин чиглүүлэлт

Процесс бүр файлтай ажиллахдаа тоон дугаар бүхий файл тодорхойлогч (file descriptor) ашигладаг. Үүнээс 0 нь stdin (оролт), 1 нь stdout (гаралт), 2 нь stderr (алдааны гаралт). Программууд өөрсдийн гаралтыг хаашаа явж байгааг мэддэггүй, зүгээр л fd 1-д бичдэг. Тиймээс shell нь child процессыг ажиллуулахаас өмнө fd 1-ийг терминалын оронд файл эсвэл pipe руу чиглүүлж өгөхөд программ өөрчлөлтгүй ажиллана. `echo data > /tmp/o.txt` үед child нь `open("/tmp/o.txt", O_WRONLY|O_CREAT|O_TRUNC)` дуудаж гарсан fd-г `dup2(fd, 1)`-ээр stdout болгон, дараа нь `execvp("echo", ...)` хийнэ.

3.4 Pipe ба олон шатлалтай холбоо

`pipe()` нь kernel дотор буфертэй, нэг чиглэлтэй байт урсгал үүсгэж хоёр fd буцаадаг: унших төгсгөл `fd[0]`, бичих төгсгөл `fd[1]`. `a | b | c` командад shell хоёр pipe үүсгэж, `a`-гийн stdout-ыг эхний pipe-ийн бичих төгсгөлтэй, `b`-ийн stdin-ыг тэр pipe-ийн унших төгсгөлтэй, `b`-ийн stdout-ыг хоёр дахь pipe-ийн бичих төгсгөлтэй, `c`-ийн stdin-ыг тэр pipe-ийн унших төгсгөлтэй холбоно (Зураг 3.1).

Энд нэг чухал нюанс бий. Pipe-ийн унших тал зөвхөн бүх бичих төгсгөл хаагдсаны дараа EOF авдаг. Хэрвээ parent болон child-ууд өөрсдөд нь хэрэггүй fd-г хаахгүй орхивол бичих төгсгөл хаагдахгүй байж, унших тал (жишээ нь `cat`) мөнхөд хүлээж гацна. Иймээс манай кодод child бүр dup2 хийсний дараа pipe-ийн fd-үүдийг хааж, parent нь ашиглаж дууссан fd-г нэн даруй хаадаг.



Зураг 3.1. `echo hi | tr a-z A-Z | cat` командын fd холболт

3.5 Дотоод (built-in) командууд

Зарим командыг child процессоор ажиллуулж болдоггүй. Жишээ нь `cd` нь ажлын хавтсыг солих тул хэрэв child-д хийвэл зөвхөн child-ийн хавтас өөрчлөгдөж, shell-д нөлөөлөхгүй. Иймээс `cd`-ийг shell өөрөө `chdir()`-ээр гүйцэтгэдэг. Манай хэрэгжилтэд `cd`, `history`, `!!`, `exit` дотоод команд бөгөөд тиймээс эдгээр нь Process tree дээр процесс үүсгэдэггүй (Зураг 6.6-д `cd /tmp` командын дугаар мод дээр байхгүй байгааг харж болно).

3.6 Zombie процесс

Child процесс дуусахад түүний нөөц ихэнх нь суллагддаг боловч процессын хүснэгтэд exit status-тайгаа хамт бичлэг үлддэг. Parent үүнийг `wait` дуудаж уншихад л бичлэг бүрэн арилна. Child дууссан ч parent хараахан `wait` хийгээгүй хугацаанд тэр процессыг zombie (`Z` төлөв) гэж нэрлэнэ. Манай shell background процессыг дараагийн prompt зурахын өмнө л цуглуулдаг учраас `sleep 5 &` гэж ажиллуулаад хэдэн секунд хүлээхэд тэр процесс zombie төлөвтэй үлдэнэ. UI үүнийг `ps` командаар OS-ээс шууд шалгаж ягаан `zombie` гэж харуулдаг (Зураг 4.3).

4. Системийн зохиомж

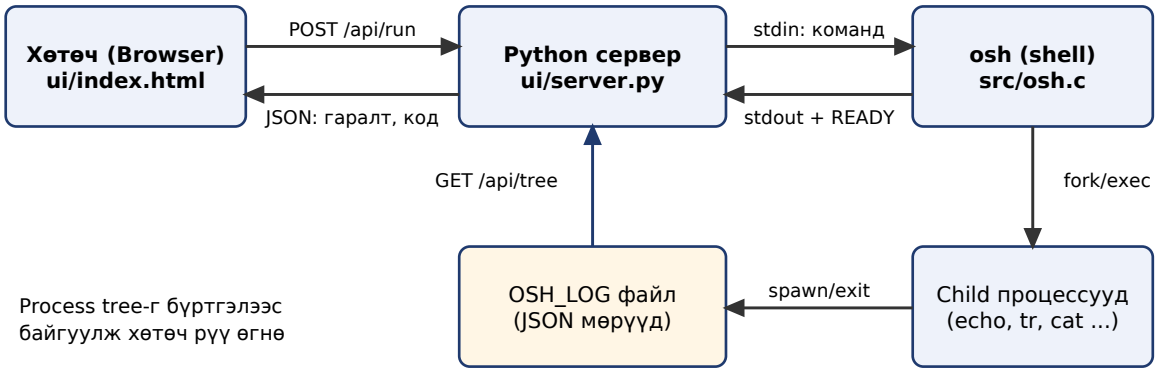
4.1 Бүрэлдэхүүн хэсгүүд

Хүснэгт 4.1. Системийн бүрэлдэхүүн

Компонент	Файл	Үүрэг
Shell	`src/osh.c`	Команд задлах, `fork`/`exec`, pipe, redirection, history, background удирдлага, үйл явдлын бүртгэл
Сервер	`ui/server.py`	Нэг `osh` session-ийг тасралтгүй барьж, HTTP API (`/api/run`, `/api/tree`, `/api/reset`) өгнө. Процессын модыг бүртгэлээс байгуулна.
UI	`ui/index.html`	Терминал, Process tree, History. 0.8 секунд тутам мод шинэчилнэ.
Тест	`tests/run_tests.sh`	22 автомат тест
Build	`Makefile`	`make`, `make test`, `make clean`

4.2 Өгөгдлийн урсгал

Хэрэглэгч UI-ийн талбарт команд бичиж Run дарахад хөтөч сервер рүү `POST /api/run` хүсэлт илгээнэ. Сервер тухайн мөрийг ажиллаж буй `osh` процессын stdin-д бичнэ. `osh` командыг гүйцэтгэж, гаралтыг stdout руу гаргаад дараагийн prompt-ийн оронд `\x01READY <код>` гэсэн тусгай мөрийг хэвлэнэ. Сервер тусдаа thread-ээр `osh`-ийн stdout-ыг уншиж энэ тэмдэгтийг олмогцоо тухайн командын гаралт болон exit code-ыг JSON хэлбэрээр хөтөч рүү буцаана. Зэрэгцээгээр хөтөч `GET /api/tree`-ээр процессын модыг асууж, сервер бүртгэлийн файлыг уншиж мод байгуулан буцаана (Зураг 4.1).



Зураг 4.1. Системийн архитектур ба өгөгдлийн урсгал

4.3 Үйл явдлын бүртгэл

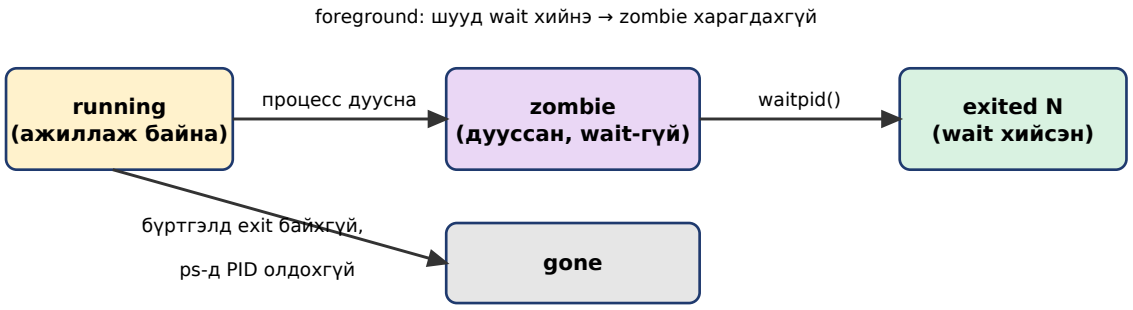
`OSH_LOG` орчны хувьсагчид файлын зам өгөгдсөн үед shell процесс үүсэх болон дуусах бүрд тэр файлд нэг JSON мөр нэмдэг. Мөр дараах талбартай:

```
{"t":1763,"ev":"spawn","pid":8128,"ppid":8179,"cmd":"ls","bg":0,"code":0,
 "seq":5,"line":"ls | wc -l"}
```

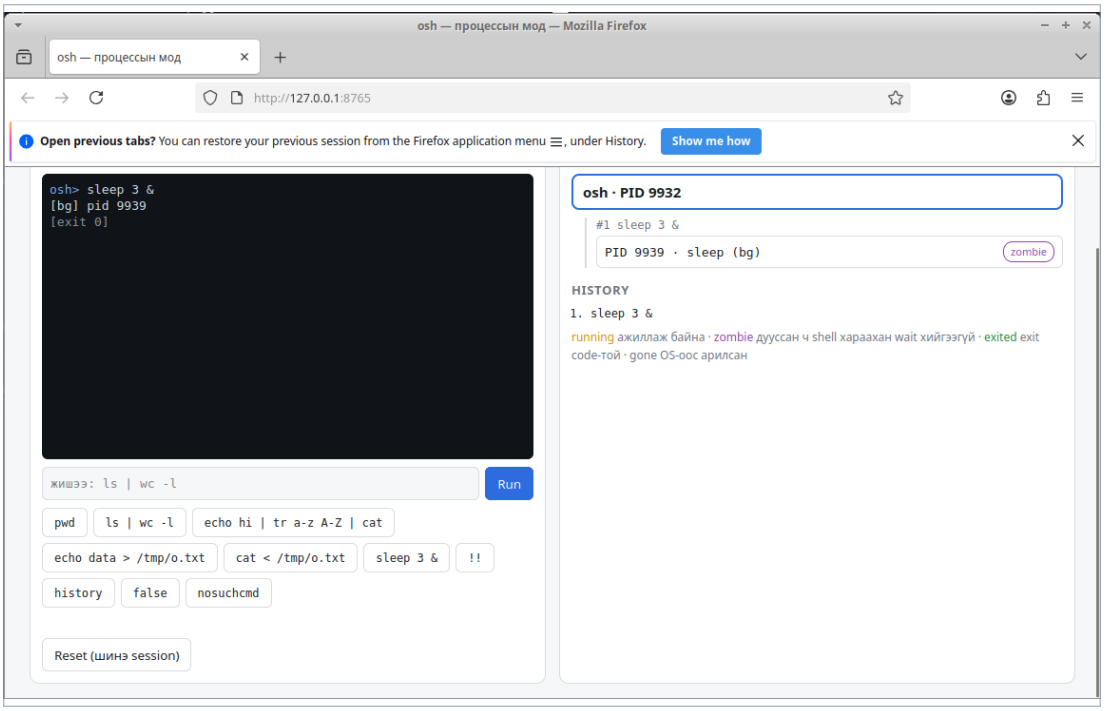
Энд `t` нь shell эхэлснээс хойших миллисекунд, `ev` нь үйл явдлын төрөл (`shell`, `spawn`, `exit`, `shell_exit`), `pid` ба `ppid` нь процесс болон эцэг процессын дугаар, `cmd` нь командын нэр, `bg` нь background эсэх, `code` нь exit код, `seq` нь session дахь командын дугаар, `line` нь бүтэн мөр. UI процессуудыг `seq`-ээр бүлэглэж нэг командын дор тэдгээр нь орох мод байгуулна. Pipe-тай командад нэг `seq` дор олон процесс гарна.

4.4 Процессын төлвийн загвар

Процесс бүр дөрвөн төлвийн нэгд байна. Child үүсэхэд `running`. Child дууссан ч shell хараахан `wait` хийгээгүй бол `zombie`. Shell `wait` хийж, бүртгэлд `exit` үйл явдал бичигдсэний дараа `exited N`. Хэрэв бүртгэлд `exit` байхгүй мөртлөө OS-ийн процессын жагсаалтад PID олдохгүй бол `gone` гэж үзнэ (Зураг 4.2). Foreground процесс ажиллаж дуусмагц shell шууд `wait` хийдэг тул тэдгээр нь zombie төлөвт харагдахгүй, зөвхөн background процесс zombie болох боломжтой.



Зураг 4.2. UI-д харагдах процессын төлвүүд



Зураг 4.3. `sleep 3 &` *дууссаны дараа, дараагийн команд өгөхөөс өмнө PID 9939 процесс ягаан* `zombie` *төлөвтэй харагдсан байдал*

5. Хэрэгжилт

5.1 Хэл, сан, орчин

Shell-ийг C11 стандартаар `gcc -O2 -Wall -Wextra -std=c11` тохиргоотойгоор хөрвүүлнэ. Зөвхөн C-ийн стандарт сан болон POSIX системийн дуудлага ашигласан, гадаад сан хэрэглээгүй. Сервер нь Python 3-ын стандарт сан (`http.server`, `subprocess`, `threading`, `json`) дээр бичигдсэн. UI нь нэг HTML файл (CSS, JavaScript дотор нь) бөгөөд гадаад сан, сүлжээ шаардахгүй. Хөтөч талд `textContent`-ээр л өгөгдөл оруулдаг тул командын текст HTML болж тайлагдахгүй.

5.2 Хязгаарууд

Статик массивын хэмжээг тогтмолоор тодорхойлсон: нэг мөр хамгийн ихдээ 1023 тэмдэгт (`MAX_LINE` = 1024), нэг командад 63 аргумент (`MAX_ARGS` = 64), pipe-ийн шатлал 16 (`MAX_STAGES`), түүх 100 команд (`MAX_HIST`), background процессын хүснэгт 64 (`MAX_JOBS`). Эдгээрийг хэтрүүлбэл буфер халихын оронд `line too long`, `too many pipe stages`, `too many arguments` гэсэн мэдээлэл өгч командыг татгалзана.

5.3 Parser

Parser нь мөрийг зай, tab болон `| < > &` тэмдэгтүүдээр token болгон задалдаг. Эдгээр дөрвөн тэмдэгт нь өөрөө тусдаа token болдог тул `echo sp>file` шиг зайгүй бичсэн ч зөв ойлгогдоно. Token бүрийг тусдаа буферт хуулдаг. Анхны хувилбарт token-ийн төгсгөлд NUL бичихийн тулд оролтын мөрийг өөрчилдөг байсан бөгөөд тэр нь `ls|wc` шиг тусгаарлагчтай залгагдсан тохиолдолд тусгаарлагч тэмдэгтийг устгаж болохыг туршилтаар олж, token-ийг тусад нь хуулдаг болгож зассан.

Syntax алдааг ялгаж мэдээлнэ: pipe-ийн өмнө эсвэл дараа команд байхгүй, `<` ба `>`-ийн дараа файлын нэр байхгүй, `&`-ийн дараа өөр үг орсон.

5.4 Командын гүйцэтгэл

`run()` функц нь задалсан шатлал бүрд `fork()` хийнэ. Child дотор эхлээд өмнөх шатлалтай холбосон pipe-ийн унших төгсгөлийг stdin-д, дараагийн шатлалтай холбосон pipe-ийн бичих төгсгөлийг stdout-д `dup2()`-ээр холбож, шаардлагагүй fd-үүдийг хаана. Дараа нь тухайн шатлалд `<` эсвэл `>` байвал файлыг нээж fd 0 эсвэл 1-ийг дахин чиглүүлнэ. Эцэст нь `execvp()` дуудна. Хэрэв үүнийг гүйцэтгэж чадахгүй бол `strerror()`-оор мэдээлж `127` кодоор гарна. Parent нь child бүрийг үүсгэх бүрд өөрт хэрэггүй болсон fd-г хааж, бүртгэлд `spawn` үйл явдал бичнэ. Foreground үед бүх child-ийг `waitpid()`-ээр хүлээж, `exit` үйл явдал бичиж, pipe-ийн сүүлийн шатлалын exit кодыг session-ийн сүүлийн статус болгон хадгална. Background үед `[bg] pid N` хэвлэж, процессыг background хүснэгтэд нэмээд хүлээлгүй буцна.

5.5 Дотоод командууд, history, background цуглуулалт

`cd` нь pipe-гүй, ганц команд байх үед shell өөрөө `chdir()` дуудна. `history` нь сүүлийн 10 командыг хамгийн сүүлийнхээс нь эхлэн дугаартай нь харуулна. `!!` нь сүүлийн командыг дэлгэцэнд давтан хэвлэж ажиллуулах бөгөөд түүх хоосон бол `No commands in history.` гэж мэдэгдэнэ. Background процессуудыг цуглуулахдаа давталт бүрийн эхэнд `waitpid(-1, &st, WNOHANG)`-ээр дууссан child байгаа эсэхийг шалгаж, олдсон бүрд `[done] pid N (cmd) exit=K` хэвлэж, бүртгэлд `exit` үйл явдал бичнэ. Энэ нь background процесс дууссаны дараа дараагийн prompt зурагдах хүртэл zombie байх шалтгаан юм.

5.6 Сервер ба session

Сервер `subprocess.Popen`-оор нэг `osh` процессыг `OSH_PROTO` ба `OSH_LOG` хувьсагчтайгаар эхлүүлж, stdin болон stdout (stderr нь stdout-тай нэгтгэсэн) хоёрыг pipe-ээр холбоно. `OSH_PROTO` тохируулсан үед shell нь prompt-ийн оронд `\x01READY <код>` мөр хэвлэж, stdout-ыг мөр бүрээр буферлэнэ. Серверийн тусдаа thread энэ стримийг уншиж командын хил заагийг олно. Хүсэлтүүд түгжээгээр дараалагддаг тул хоёр команд зэрэг ажиллахгүй. Хэрэв `osh` процесс (жишээ нь `exit` командаар) дууссан бол дараагийн хүсэлтээр сервер шинэ session эхлүүлнэ. Нэг session барьдаг тул `cd /tmp` хийсний дараах `pwd` нь `/tmp`-г харуулна.

Серверийн аюулгүй байдлын тал: сервер зөвхөн `127.0.0.1`-д бүрэн нээгддэг, `Host` толгойг шалгаж `127.0.0.1:8765` болон `localhost:8765`-аас бусдыг татгалзана, команд 1000 тэмдэгтээс уртыг зөвшөөрөхгүй, нэг командыг 20 секундээс удаан

хүлээхгүй. Ингэсэн ч сервер нь бодит shell команд ажиллуулдаг тул зөвхөн итгэлтэй локал орчинд ашиглах ёстой.

5.7 UI-ийн хэрэгжилт

UI нь зүүн талд терминал (гаралт, `[exit N]`), команд оруулах талбар, бэлэн командын товчууд, Reset товчтой. Баруун талд Process tree болон History байна. Мод дээр shell нь хамгийн дээд зангилаа, түүний доор сүүлийн 12 команд `#дугаар` болон бүтэн мөрөөрөө, тэдгээрийн доор процесс бүр PID, командын нэр, төлөвтэй харагдана. Төлөв бүр өнгөтэй: `running` шар, `zombie` ягаан, `exited 0` ногоон, `exited` өөр код улаан, `gone` саарал. Хуудас light болон dark горимыг дэмжинэ.

6. Туршилт ба UI дүрслэл

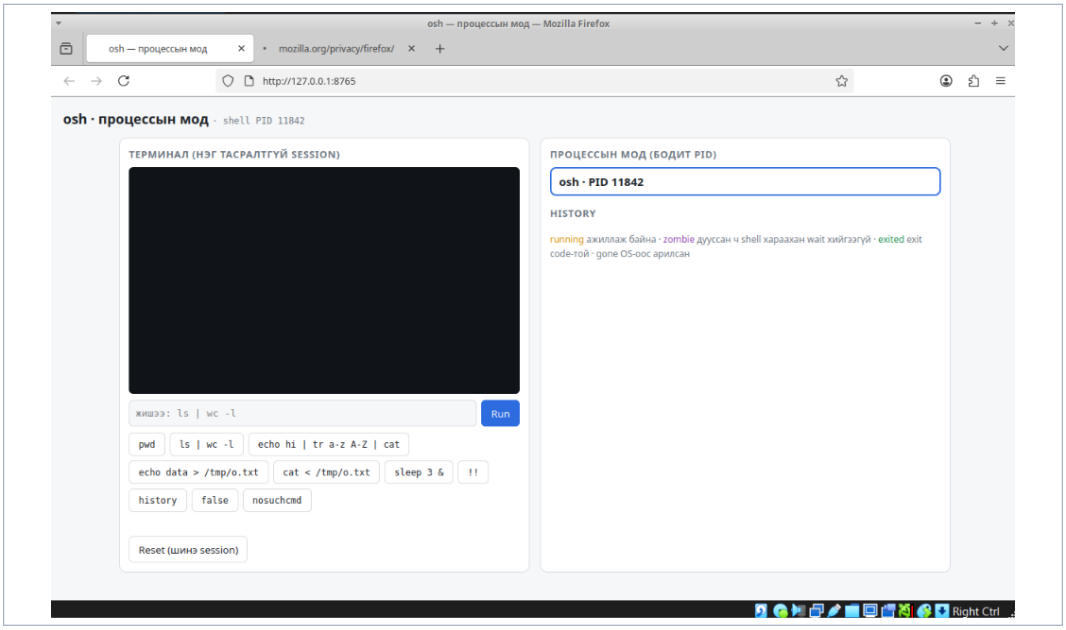
6.1 Юуг өөрчилж турших вэ

UI дээр хэрэглэгч дараах зүйлсийг өөрчилж дахин туршиж болно. Дурын команд, аргумент бичнэ. Pipe-ийн шатлалын тоог 1-ээс 16 хүртэл өөрчилнө. Redirection-д өөр файл ашиглана. Background командын хугацааг `sleep N &`-ийн `N`-ээр тохируулна. `!!` болон `history`-ээр түүхийг давтана. Reset товчоор shell-ийг шинээр эхлүүлж мод болон терминалыг цэвэрлэнэ. Бэлэн товчнууд нь түгээмэл жишээнүүдийг нэг даралтаар оруулж өгнө.

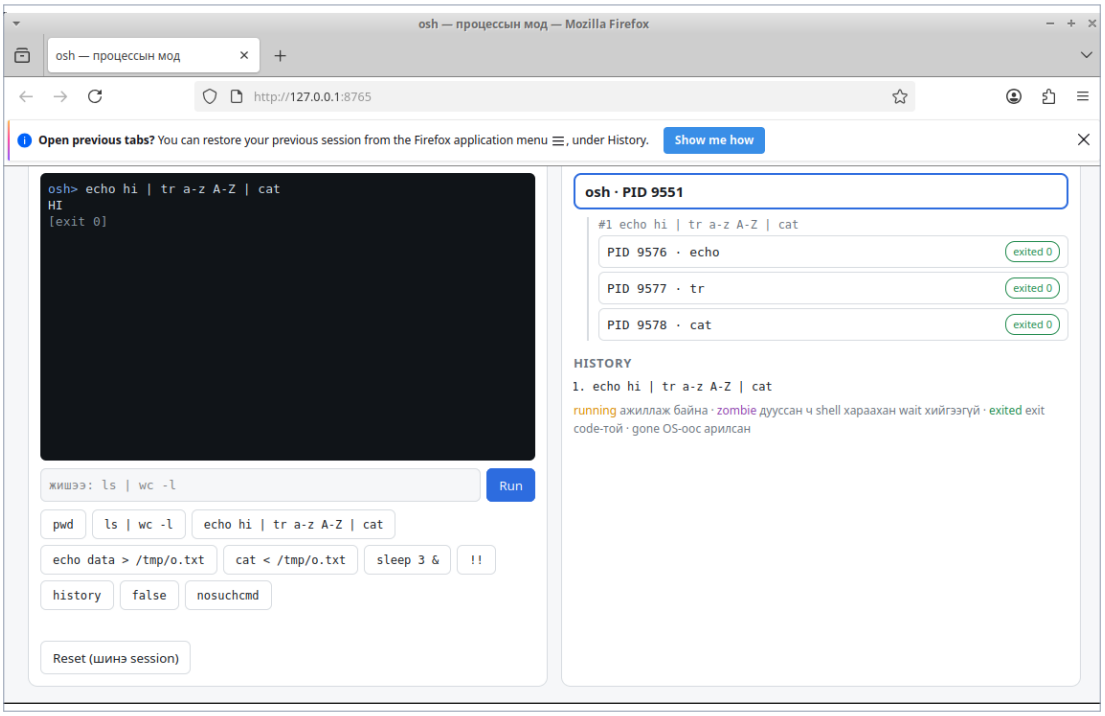
6.2 Юуг ажиглах вэ

Командыг ажиллуулсны дараа терминалд гаралт болон `[exit N]` код гарна. Баруун талын модонд тухайн командын дор үүссэн процессууд бодит PID-тайгаа харагдана. Pipe-тай командад нэг команд дор олон процесс гарна. Background командад `(bg)` тэмдэг гарч, эхлээд `running`, дууссаны дараа shell `wait` хийх хүртэл `zombie`, түүний дараа `exited 0` болно. `cd` зэрэг дотоод команд процесс үүсгэхгүй. Алдаатай командад улаан `exited` төлөв гарна.

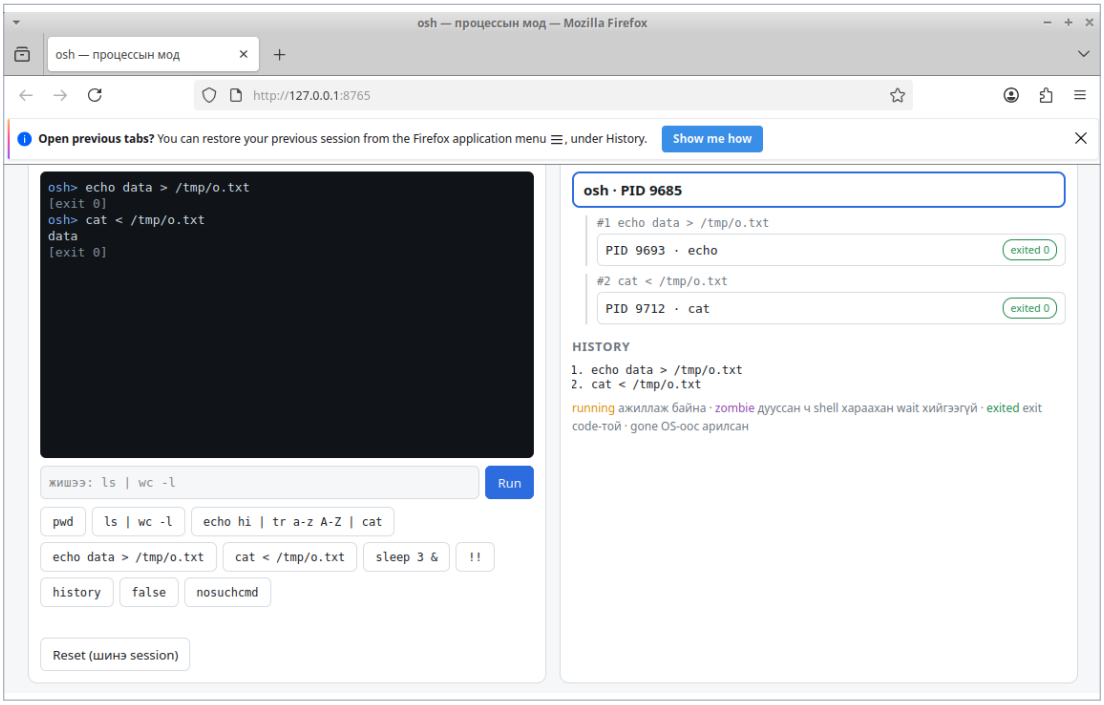
6.3 UI-ийн бодит дэлгэцийн зураг



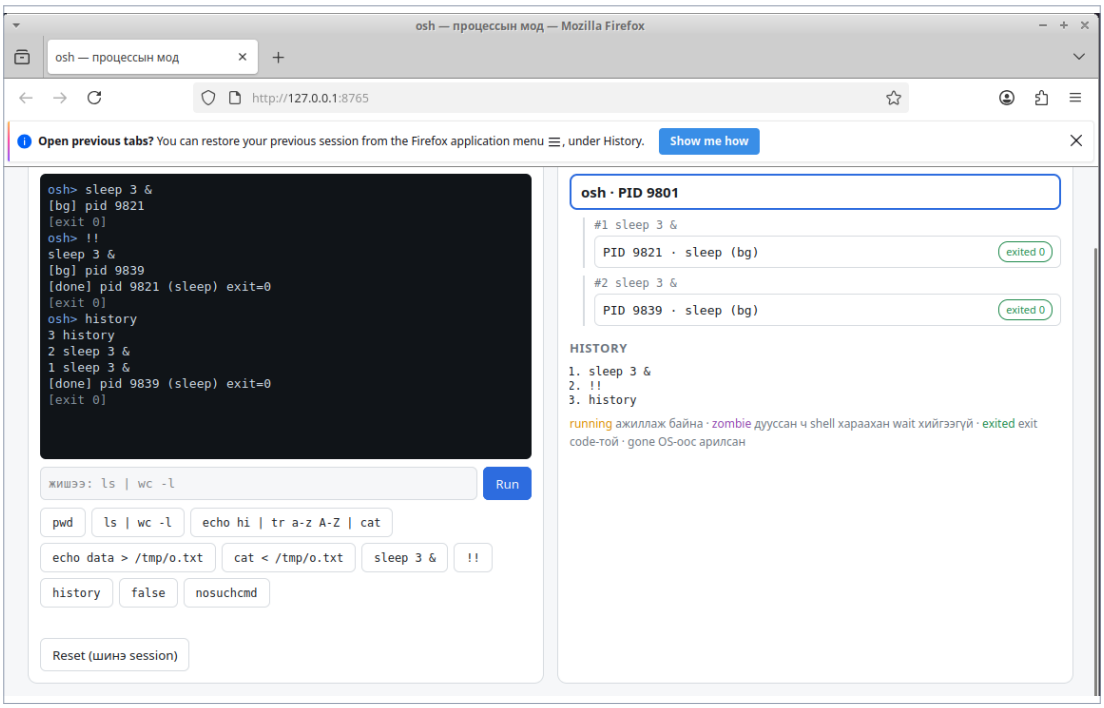
Зураг 6.1. Эхлэх үеийн UI. Зөвхөн shell-ийн PID, хоосон терминал, бэлэн командын товчууд харагдаж байна



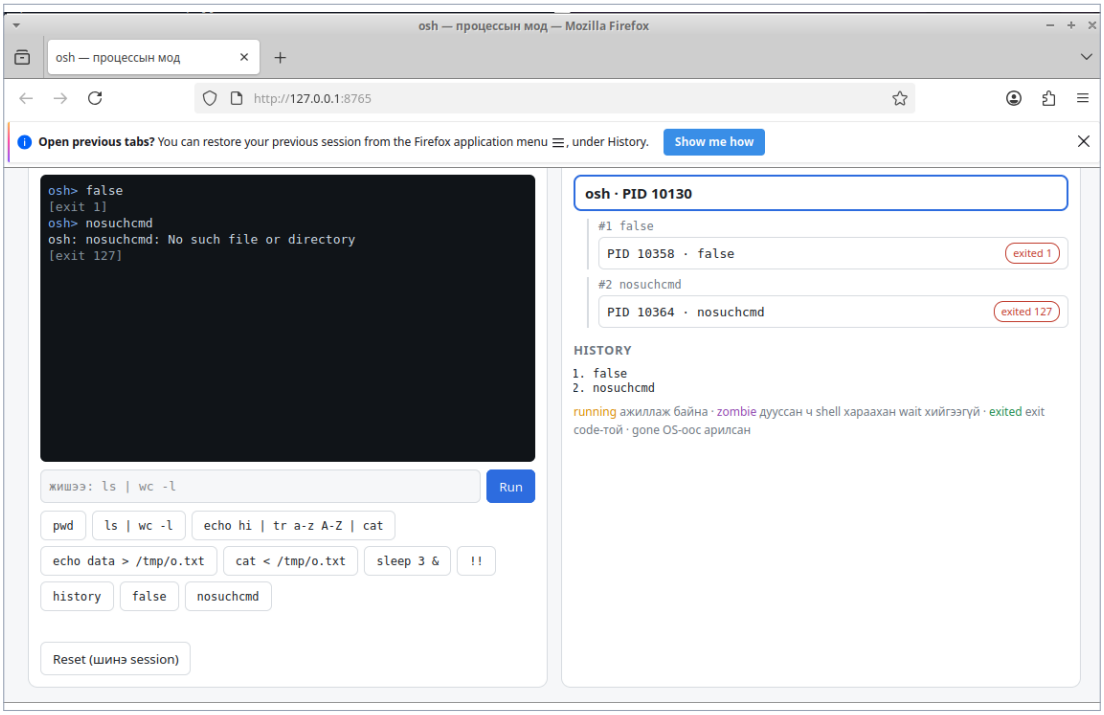
Зураг 6.2. `echo hi | tr a-z A-Z | cat`*. Нэг команд дор гурван процесс (PID 9576, 9577, 9578) харагдаж байна*



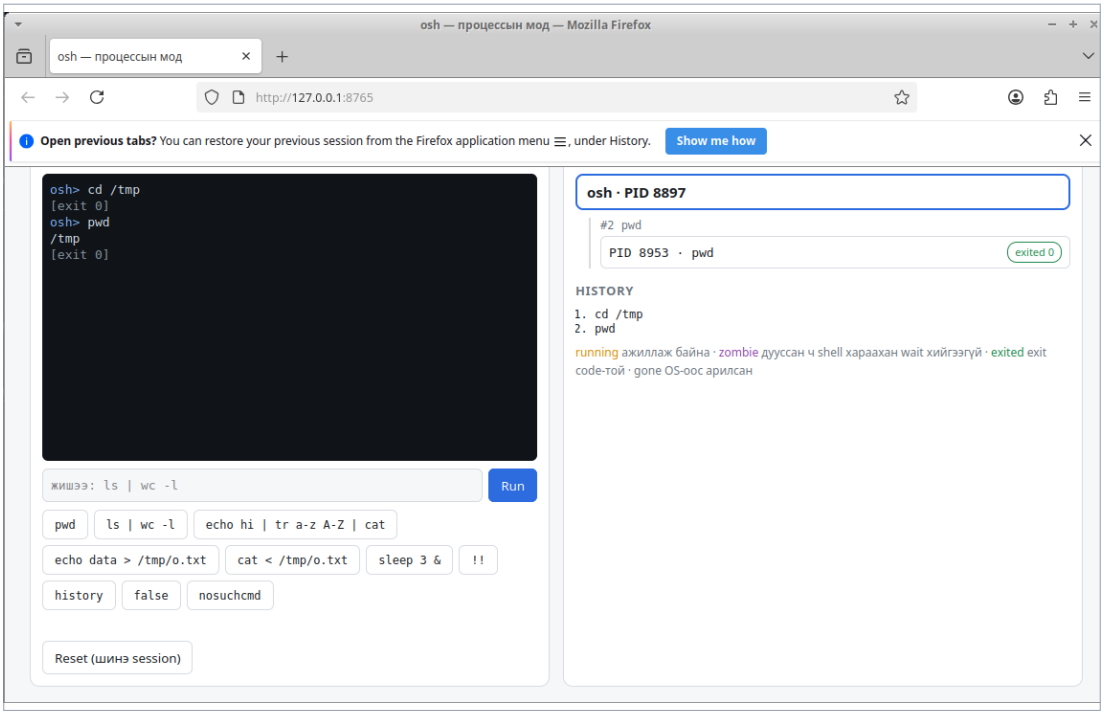
Зураг 6.3. `echo data > /tmp/o.txt`, *дараа нь* `cat < /tmp/o.txt`. *Хоёр тусдаа команд, тус бүр нэг процесс*



Зураг 6.4. `sleep 3 &`, `!!`, `history`. `[done]` *мессеж нь shell background процессыг дараагийн prompt-ийн өмнө цуглуулсныг харуулна*



Зураг 6.5. `false` *(улаан* `exited 1`*) ба* `nosuchcmd` *(*`No such file or directory`*, улаан* `exited 127`*)*



Зураг 6.6. `cd /tmp` *дараа нь* `pwd` *нь* `/tmp` *харуулсан. Модонд зөвхөн* `#2 pwd` *байгаа нь* `cd` *дотоод команд тул процесс үүсгээгүйг харуулна*

7. Туршилтын арга

7.1 Орчин

Хүснэгт 7.1. Туршилтын орчин

Зүйл	Утга
Үйлдлийн систем	Xubuntu 25.10 (VirtualBox виртуал машин), Linux 6.17.0-5-generic, x86_64
Compiler	gcc 15.2.0 (Ubuntu 15.2.0-4ubuntu4), `-O2 -Wall -Wextra -std=c11`
Build	GNU Make 4.4.1
Python	3.13.7
Хөтөч (UI)	Mozilla Firefox (Xubuntu 25.10-д суусан хувилбар)
Нөөц	VM-д 1 CPU. Програмын ачаалал бага тул хангалттай

7.2 Автомат тестийн арга

Автомат тестүүд `tests/run_tests.sh` дотор bash скриптээр бичигдсэн. Тест бүр `osh`-ийн stdin-д командыг `printf`-ээр дамжуулж, гаралт (stdout, шаардлагатай бол stderr)-д хүлээгдсэн текст агуулагдаж байгааг шалгана. Redirection-ийн тестүүд `mktemp -d`-ээр үүсгэсэн түр хавтас дотор ажиллах тул төслийн файлд нөлөөлөхгүй. Бүртгэлийн тестүүд `OSH_LOG` файлд `spawn` болон `exit` үйл явдал бичигдсэнийг шалгана. Бүх тест тэнцвэл скрипт `RESULT: 22 passed, 0 failed` гэж хэвлээд 0 кодоор гарна, нэг нь унавал `FAIL` мөр хэвлэж 0-ээс өөр кодоор гарна. `make test` нь эхлээд `osh`-ийг хөрвүүлж, дараа нь энэ скриптийг ажиллуулна.

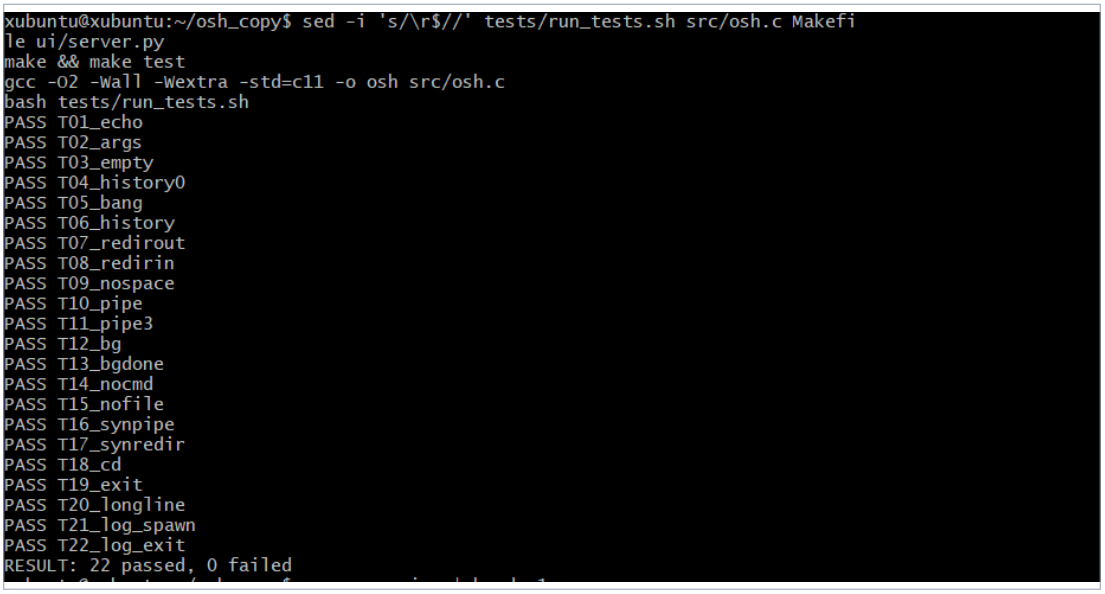
7.3 Гар туршилтын арга

UI болон серверийг автомат тест хийгээгүй тул хөтөч дээр командуудыг гараар оруулж, терминалын гаралт, exit код, мод дээрх PID ба төлвийг нүдээр шалгасан. Ажиглалтыг screenshot-оор баримтжуулсан (Зураг 6.1–6.6).

8. Үр дүн

8.1 Автомат тестийн үр дүн

Xubuntu 25.10 дээр `make && make test` ажиллуулахад хөрвүүлэлтэд анхааруулга гараагүй бөгөөд 22 тест бүгд тэнцсэн (Зураг 8.1).



Зураг 8.1. `make test`*: T01–T22 бүгд PASS,* `RESULT: 22 passed, 0 failed`

Хүснэгт 8.1. Автомат тестүүд

Тест	Оролт	Шалгах зүйл	Үр дүн
T01 echo	`echo hello`	Энгийн командын гаралт	PASS
T02 args	`echo a b c`	Олон аргумент	PASS
T03 empty	хоосон мөр, зайтай мөр	Хоосон оролтод унахгүй, дараагийн команд ажиллана	PASS
T04 history0	`!!`	Түүх хоосон үед `No commands in history.`	PASS
T05 bang	`echo hi`, `!!`	Сүүлийн команд давтагдана (`hi` хоёр удаа)	PASS
T06 history	`echo a`, `echo b`, `history`	Дугаартай жагсаалт (`2 echo b`)	PASS
T07 redirout	`echo data > файл`, `cat файл`	Гаралтыг файлд чиглүүлэх	PASS
T08 redirin	`cat < файл`	Файлаас оролт авах	PASS
T09 nospace	`echo sp>файл`, `cat<файл`	Зайгүй бичигдсэн чиглүүлэлт	PASS
T10 pipe	`printf ... | wc -l`	2 шатлалтай pipe	PASS
T11 pipe3	`echo b | tr b B | cat`	3 шатлалтай pipe (`B`)	PASS
T12 bg	`sleep 0.2 &`	`[bg] pid` хэвлэгдэнэ	PASS
T13 bgdone	`sleep 0.1 &`, `sleep 0.5`, `echo x`	`[done]` мессеж гарна	PASS
T14 nocmd	`nosuchcmd_zz`	`No such file or directory`	PASS
T15 nofile	`cat < nofile_zz`	Байхгүй файлын алдаа	PASS
T16 synpipe	`| ls`	`syntax error`	PASS
T17 synredir	`ls >`	`missing file name`	PASS
T18 cd	`cd <хавтас>`, `pwd`	Ажлын хавтас солигдоно	PASS
T19 exit	`echo before`, `exit`, `echo after`	`exit`-ийн дараа команд ажиллахгүй	PASS
T20 longline	3000 тэмдэгтийн мөр	`line too long`	PASS
T21 log_spawn	`echo x | cat` + `OSH_LOG`	Бүртгэлд `"ev":"spawn"` бий	PASS
T22 log_exit	ижил	Бүртгэлд `"ev":"exit"` бий	PASS

8.2 Гар туршилтын үр дүн

Хүснэгт 8.2-т UI дээр гараар туршсан командуудын бодит үр дүнг screenshot-оос авч харуулав. PID-ууд нь тухайн ажиллуулалтын утгууд тул дахин ажиллуулахад өөр байна.

Хүснэгт 8.2. Гар туршилтын үр дүн

№	Команд	Хүлээгдсэн	Ажигласан	Зураг
1	`echo hi | tr a-z A-Z | cat`	Гаралт `HI`, 3 процесс	`HI`, exit 0. PID 9576 (echo), 9577 (tr), 9578 (cat), бүгд exited 0	6.2
2	`echo data > /tmp/o.txt`	Гаралтгүй, exit 0	`[exit 0]`, PID 9693 exited 0	6.3
3	`cat < /tmp/o.txt`	`data`	`data`, PID 9712 exited 0	6.3
4	`sleep 3 &`	`[bg] pid`, shell шууд буцна	`[bg] pid 9821`, дараа нь `[done] pid 9821 (sleep) exit=0`	6.4
5	`!!`	`sleep 3 &` давтагдана	Дэлгэцэнд `sleep 3 &` хэвлэгдэж `[bg] pid 9839`. Мод дээр #2 болж нэмэгдэнэ	6.4
6	`history`	Дугаартай жагсаалт	`3 history`, `2 sleep 3 &`, `1 sleep 3 &`	6.4
7	`false`	exit 1	Улаан `exited 1` (PID 10358)	6.5
8	`nosuchcmd`	Алдаа, exit 127	`No such file or directory`, улаан `exited 127` (PID 10364)	6.5
9	`cd /tmp`, `pwd`	`/tmp`, `cd` мод дээр процесс биш	`/tmp`. Модонд зөвхөн `#2 pwd` (PID 8953)	6.6
10	`sleep 3 &`, дараа нь команд өгөхгүй хүлээх	Дууссан процесс zombie болно	PID 9939 ягаан `zombie` төлөвтэй. Терминалд `[bg] pid 9939`	4.3

8.3 Нэмэлт туршилт (захын тохиолдол)

Автомат тестээс гадна дараах захын тохиолдлуудыг терминалд `./osh` дээр гараар ажиллуулж, гаралтыг Хүснэгт 8.3-т нэгтгэв. Эдгээрийг автомат тест болгох нь цаашдын ажил.

Хүснэгт 8.3. Нэмэлт туршилт

№	Оролт	Хүлээгдсэн	Ажигласан
1	`echo abc | cat | cat | cat | wc -c`	4 шатлалтай pipe, `4`	`4`
2	16 шатлалтай pipe (`echo z | cat` … 15 удаа)	Дээд хязгаарт ажиллана	`z`
3	17 шатлалтай pipe	Татгалзана	`osh: too many pipe stages (max 16)`
4	`sleep 1 & ls`	Татгалзана	`osh: '&' must be last`
5	70 аргументтай `echo`	Татгалзана	`osh: too many arguments`
6	`echo hello world | wc -w > файл`, `cat файл`	Pipe ба redirection хамт	`2`
7	`nosuchcmd`, дараа нь `!!`	Алдаатай командыг давтана	Хоёр удаа `No such file or directory`
8	`cd /no_such_dir`, `pwd`	Алдаа гарч, хавтас өөрчлөгдөхгүй	`osh: cd: No such file or directory`, `pwd` өмнөх хавтас
9	`echo x > f1 > f2`	Хоёр файл (bash шиг)	Зөвхөн `f2` үүсч `x` бичигдсэн. `f1` үүсээгүй. Сүүлийн чиглүүлэлт давамгайлна (хязгаарлалт)

9. Хэлэлцүүлэг

9.1 Үр дүнгийн тайлбар

22 автомат тест бүгд тэнцсэн нь тестэд тодорхойлсон оролтод shell хүлээгдсэн гаралт гаргаж байгааг харуулна. Тестүүд нь үндсэн боломжуудаас гадна буруу болон захын оролт (хоосон мөр, хэт урт мөр, синтакс алдаа, байхгүй файл) болон `OSH_LOG` бүртгэлийг хамарсан. UI дээрх гар туршилт нь Process tree-д командын дугаар, процесс бүрийн PID, төлөв, exit код нь терминалын гаралттай нийцэж байгааг баталсан. Pipe-тай командад нэг команд дор гурван процесс гарсан нь `fork` хийгдсэн тоо болон онолын загвартай таарна.

Нэг session барьдаг загвар нь ажиллаж байгаа shell-ийн төлөвийг (ажлын хавтас, түүх, background процессууд) командуудын хооронд хадгалдаг. Үүний үр дүнд `cd /tmp` дараа нь `pwd` нь `/tmp` харуулж, `!!` нь өмнөх UI командыг давтаж чадна (Зураг 6.4, 6.6). Background процесс дууссаны дараа shell `wait` хийх хүртэл `zombie` төлөвтэй харагдах нь онолын 3.6 хэсгийн ойлголтыг ажиглагдахуйц болгосон.

9.2 Хязгаарлалт

Хүснэгт 9.1. Хязгаарлалт

Хязгаарлалт	Тайлбар
Хэрэгжээгүй синтакс	Хашилт (`"a b"`), wildcard (`*`), орчны хувьсагч (`$HOME`), `>>`, `2>`, `!n`, `;`, `&&` дэмжигдэхгүй. `&` зөвхөн мөрийн төгсгөлд.
Background-ийн статус	Background процессын exit код нь shell дараагийн prompt-ийн өмнө цуглуулахад л бүртгэгдэнэ, тиймээс командгүй хүлээх үед zombie хэвээр харагдана.
Үйл явдлын бүртгэл	Process tree нь shell-ийн бичсэн бүртгэл дээр суурилсан. Зөвхөн shell-ийн үүсгэсэн процессыг агуулна. Төлвийг нэмж `ps`-ээр шалгадаг.
Тестийн хамрах хүрээ	UI болон сервер автомат тестээр хамрагдаагүй, гараар шалгасан. 22/22 нь зөвхөн тестэд хамрагдсан нөхцөлийн баталгаа.
Нэг session	Сервер нэг `osh` барьдаг тул хоёр хэрэглэгч нэг session-ийг хуваалцана. Хүсэлт дарааллаар гүйцэтгэгддэг.
Орчин	Windows дээр native ажиллахгүй (`fork`, `pipe` POSIX). WSL эсвэл Linux/macOS хэрэгтэй. Зөвхөн Linux (Xubuntu) дээр туршсан, macOS-д туршаагүй.
Олон чиглүүлэлт	Нэг шатлалд `>` хоёр удаа бичвэл зөвхөн сүүлийнх нь ажиллана, эхний файл үүсэхгүй (bash хоёуланг үүсгэдэг).
Туршилтын явцад гарсан асуудал	Виртуал машин дээрх эхний ажиллуулалтад UI-ийн команд нэг удаа хариу өгөхгүй гацсан. Сервер, shell-ийг дахин эхлүүлсний дараа хэвийн ажилласан. Шалтгааныг бүрэн тогтоогоогүй.

10. Дүгнэлт

Operating System Concepts номын UNIX Shell төслийн үндсэн шаардлагыг (`fork`/`execvp`/`waitpid`, `&`, `!!`, `< >`, `|`) C хэл дээр бүрэн хэрэгжүүлж, нэмэлтээр хэдэн ч шатлалтай pipe, `cd`, `history` дугаарласан жагсаалт, ойлгомжтой алдааны мэдээллийг оруулсан. Process tree UI нь shell-ийн бодит үйл явдлын бүртгэл болон OS-ийн процессын төлвөөс байгуулагддаг тул дүрслэл нь бодит ажиллагаатай шууд холбогдсон. Xubuntu 25.10 дээр 22 автомат тест бүгд тэнцэж, UI-ийн гар туршилтыг screenshot-оор баримтжуулсан.

Энэ ажлаас дараах зүйлийг ойлгосон. Fork, exec, wait гурав нь хоорондоо хэрхэн уялддаг. Дахин чиглүүлэлт нь программд мэдэгдэлгүйгээр fd-ийг солих замаар хийгддэг. Pipe-д хэрэггүй fd-г хаахгүй бол унших тал гацдаг. `cd` зэрэг командыг shell өөрөө гүйцэтгэх ёстой. Zombie нь `wait` хийгдээгүй child-ийн бичлэг. Цаашид хашилт, wildcard, `>>` ба `2>`, бүрэн job control (`fg`, `bg`) нэмж, background процессын exit кодыг дуусах агшинд нь SIGCHLD хэндлерээр бүртгэж, UI болон серверт автомат тест бичих боломжтой.

Ашигласан материал

1. A. Silberschatz, P. B. Galvin, G. Gagne. Operating System Concepts, 10th edition. Wiley, 2018. Chapter 3, Programming Project 1: UNIX Shell.

2. Linux manual pages: fork(2), execvp(3), waitpid(2), pipe(2), dup2(2), open(2), chdir(2).

3. Python Software Foundation. Python 3 documentation: subprocess, http.server, threading.

4. GNU Make Manual: https://www.gnu.org/software/make/manual/

5. Хиймэл оюун ашигласан тухай: Claude (Anthropic)-ийг эх код (`src/osh.c`, `ui/`), автомат тест, тайлан ба manual-ын ноорог бэлтгэхэд ашигласан. Багийн гишүүд кодыг уншиж, өөрсдийн компьютерт ажиллуулж, тест болон UI-г туршиж, тайланг шалгаж засварласан.

6. Төслийн эх код: https://github.com/StrAVengeR/IV-2-1-UNIX-Shell

Хавсралт А. Ажлын төлөвлөгөө

Талбар	Мэдээлэл
Хуваарилалтын код	IV-2-1
Төсөл	UNIX Shell
Repository	https://github.com/StrAVengeR/IV-2-1-UNIX-Shell
Хэл/сан/орчин	C11 (gcc), POSIX, Python 3 (стандарт сан), HTML/CSS/JavaScript
Үндсэн дүрслэл	Process tree (PID, командын дугаар, төлөв)

Хүснэгт А.1. Туршилтын төлөвлөгөө

№	Оролт / нөхцөл	Хүлээгдсэн үр дүн
1	`make`; `make test`	Анхааруулгагүй build; `RESULT: 22 passed, 0 failed`
2	`echo hi | tr a-z A-Z | cat`	`HI`; нэг команд дор 3 процесс
3	`echo data > /tmp/o.txt`; `cat < /tmp/o.txt`	`data`; тус бүр нэг процесс
4	`sleep 3 &`; команд өгөхгүй хүлээх; дараа нь дурын команд	`running` → `zombie` → `exited 0`
5	`false`; `nosuchcmd`	`exited 1`; `No such file or directory`, `exited 127`
6	`cd /tmp`; `pwd`	`/tmp`; `cd` процесс үүсгэхгүй
7	17 шатлалтай pipe; `sleep 1 & ls`; 70 аргумент	`too many pipe stages`; `'&' must be last`; `too many arguments`